# Longest String Chain

AlgoExpert · Very Hard · Dynamic Programming

# The problem

- A chain is built by removing one character at a time where every intermediate string is in the input.
- Return the longest chain (length >= 2) from longest to shortest, or [].

# Intuition

- Process strings by length; each string's chain extends the best chain of any one-character-shorter string.

# Brute force vs optimal

- Compare all pairs for one-deletion → $O(n^2 \cdot L)$
- Sort by length + hash map, try each deletion → $O(n \cdot L^2)$

# Approach

- 1. Sort by length
- 2. For each string, try removing each character
- 3. Look up the result
- 4. Keep best lengths and next links

# Worked example

| string (by length) | chain length |
|---|---|
| ae | 1 |
| abc | 1 |
| abd | 1 |
| ade | 2 |
| abde | 3 |
| abcde | 4 |
| 1abde | 4 |
| abcdef | 5 |

Process by length; try deleting each char

# Worked example

`abcdef → abcde → abde → ade → ae`

Longest chain: 5

# Java solution

```java
public class LongestStringChain {
    public static List<String> longestStringChain(List<String> strings) {
        List<String> sorted = new ArrayList<>(strings);
        sorted.sort(Comparator.comparingInt(String::length));
        Map<String, Integer> len = new HashMap<>();
        Map<String, String> next = new HashMap<>();
        String bestStart = null;
        for (String s : sorted) {
            len.put(s, 1);
            for (int i = 0; i < s.length(); i++) {
                String shorter = s.substring(0, i) + s.substring(i + 1);
                if (len.containsKey(shorter) && len.get(shorter) + 1 > len.get(s)) { len.put(s, len.get(shorter) + 1); next.put(s, shorter); }
            }
            if (bestStart == null || len.get(s) > len.get(bestStart)) bestStart = s;
        }
        List<String> out = new ArrayList<>();
        if (bestStart == null || len.get(bestStart) < 2) return out;
        for (String s = bestStart; s != null; s = next.get(s)) out.add(s);
        return out;
    }
}
```

# Complexity

- Time: O(n * m^2 + n log n)
- Space: O(n * m)

# Edge cases & pitfalls

- Chain must have ≥ 2 strings
- Track next for reconstruction
- Duplicates

# Interview tips & follow-ups

- Relate to LIS

# Recap

- Process strings by length; each string's chain extends the best chain of any one-character-shorter string.
- Complexity: O(n * m^2 + n log n)